

explicit(bool)

Contents

- [Motivation](#)
- [Proposal](#)
- [Wording](#)
- [Acknowledgements](#)

Motivation

When writing a class template which wraps a member of template parameter type, it's useful to expose constructors that allow the user to construct the member in place. In order to do this properly, fulfilling the usage that the class designer expects, such constructors need to be `explicit` or not based on whether the corresponding class's constructor is `explicit` or not. Conditionally explicit constructors appear throughout the standard library in some of the most commonly used utility types - `std::pair`, `std::tuple`, and now `std::optional` and `std::variant` too, among others. This permits very natural code:

```
pair<string, string> safe() {
    return {"meow", "purrr"}; // ok
}

pair<vector<int>, vector<int>> unsafe() {
    return {11, 22}; // error
}
```

Despite being very useful functionality, it is quite cumbersome to implement. While conceptually we talk about a constructor being "conditionally `explicit`," the only way to implement that functionality is to write two constructors that are mutually disjoint - both of which beyond that do exactly the same thing. This gets a little better with concepts, where at least we don't have to duplicate the convertibility trait, but either way we have two duplicated constructors, and you just have to know why they're written the way they are. Here is an example from `std::pair`:

C++17 today (SFINAE)	With Concepts
<pre>template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2> , int> = 0> constexpr pair(U1&&, U2&&); template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && !(std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2>) , int> = 0> ; };</pre>	<pre>template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2> requires std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2> constexpr pair(U1&&, U2&&); template <typename U1=T1, typename U2=T2> requires std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> explicit constexpr pair(U1&&, U2&&); };</pre>

```
explicit constexpr pair(U1&&, U2&& );
};
```

These aren't conceptually two constructors and they aren't meaningfully two constructors. The intent of the design is to have one single, conditionally explicit constructor. While this trick works, it's a fairly tedious, verbose, repetitive solution. How many third party libraries have code that *should* follow this idiom but doesn't due to the difficulty? Let's do better.

Proposal

The proposal is simply to allow for the direct declaration of conditionally `explicit` constructors (and conversion operators) in the same way that we currently specify that a function is conditionally `noexcept`: with an extra boolean constant argument. That would allow the above example to be written much more directly as:

SFINAE	SFINAE + explicit(bool)
<pre>template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2> , int> = 0> constexpr pair(U1&&, U2&&); template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && !(std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2>) , int> = 0> explicit constexpr pair(U1&&, U2&&); };</pre>	<pre>template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> , int> = 0> explicit(!std::is_convertible_v<U1, T1> !std::is_convertible_v<U2, T2>) constexpr pair(U1&&, U2&&); };</pre>
Concepts	Concepts + explicit(bool)
<pre>template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2> requires std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2> constexpr pair(U1&&, U2&&); template <typename U1=T1, typename U2=T2> requires std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> explicit constexpr pair(U1&&, U2&&); };</pre>	<pre>template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2> requires std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> explicit(!std::is_convertible_v<U1, T1> !std::is_convertible_v<U2, T2>) constexpr pair(U1&&, U2&&); };</pre>

In both cases, the constructor at the right is `explicit` if the condition in parentheses is `true`, otherwise it's not. *One* truly, conditionally explicit constructor - finally a direct way to express our intent.

This is a pure language extension, such syntax is ill-formed today, and no existing code will break.

This proposal also suggests the creation of a feature-test macro for this language feature: `__cpp_explicit_bool`.

Proposed Wording

Language

In 10.1.2 [dcl.fct.spec] paragraph 1:

Function-specifiers can be used only in function declarations.

function-specifier:

`virtual`

~~`explicit`~~

`explicit-specifier`

`explicit-specifier`:

`explicit` (`constant-expression`).

`explicit`

In 10.1.2 [dcl.fct.spec] paragraph 3:

~~The `explicit` specifier~~An `explicit-specifier` shall be used only in the declaration of a constructor or conversion function within its class definition [...]

Insert new paragraph after 10.1.2 [dcl.fct.spec] paragraph 3:

In an `explicit-specifier`, the `constant-expression`, if supplied, shall be a contextually converted constant expression of type `bool` (`(expr.const)`). The `explicit-specifier` `explicit` without a `constant-expression` is equivalent to the `explicit-specifier` `explicit(true)`. If the constant expression evaluates to true, the function is explicit. Otherwise, the function is not explicit. A `(.` token that follows `explicit` is parsed as part of the `explicit-specifier`.

In 11.3 [dcl.meaning], paragraph 2:

A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, ~~`explicit`~~, or `typedef` specifier or an `explicit-specifier` applies directly to each *declarator-id* in an *init-declarator-list* or *member-declarator-list*; [...]

In 11.6.1 [dcl.init.aggr], paragraph 1, change explicit to not use code font:

An aggregate is an array or a class with

- no user-provided, ~~`explicit`~~ `explicit`, or inherited constructors (15.1 [class.ctor]),
- [...]

In 15.1 [class.ctor], paragraph 1:

In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be `friend`, `inline`, ~~`explicit`~~, or `constexpr` or an `explicit-specifier`.

In 15.3.1 [class.conv.ctor], paragraph 1:

A constructor that is not explicit (10.1.2 [dcl.fct.spec]) ~~declared without the function-specifier `explicit`~~ specifies a conversion from the types of its parameters (if any) to the type of its class. Such a constructor is called a *converting constructor*.

Insert into 16.3.1 [over.match.funcs], paragraph 7, explaining how to handle value-dependent *explicit-specifiers*:

In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction ([temp.over], [temp.deduct]). If a constructor template or conversion function template has an `explicit-specifier` whose `constant-expression` is value-dependent ([temp.dep]), template argument deduction is performed first and then, if the context requires a candidate that is not explicit and the generated specialization is explicit ([dcl.fct.spec]), it will be removed from the candidate set. Those candidates are then handled as candidate functions in the usual way.¹²⁵ A given name can refer to one or more function templates and also to a set of overloaded non-template functions. In such a case, the candidate functions generated from each function template are combined with the set of non-template candidate functions.

In 16.3.1.7 [over.match.list], paragraph 1, change explicit to not use code font:

If the initializer list has no elements and `T` has a default constructor, the first phase is omitted. In copy-list-initialization, if an ~~`explicit`~~ `explicit` constructor is chosen, the initialization is ill-formed.

In 16.3.1.8 [over.match.class.deduct], paragraph 2, rewrite:

~~Each such notional constructor is considered to be explicit if the function or function template was generated from a constructor or deduction-guide that was declared `explicit`.~~ If the function or function template was generated from a constructor or deduction-

guide that had an *explicit-specifier*, each such notional constructor is considered to have that same *explicit-specifier*.

In 17.9.2 [temp.deduct], insert an extra case for SFINAE:

Only invalid types and expressions in the immediate context of the function type ~~and~~, its template parameter types, and its *explicit-specifier* can result in a deduction failure. [Note: The substitution into types and expressions can result in effects such as the instantiation of class template specializations and/or function template specializations, the generation of implicitly-defined functions, etc. Such effects are not in the “immediate context” and can result in the program being ill-formed. —end note]

Library

In 20.4.2.2 [functions.within.classes], remove paragraph 2:

~~For the sake of exposition, the library clauses sometimes annotate constructors with `EXPLICIT`. Such a constructor is conditionally declared as either explicit or non-explicit (15.3.1 [class.conv.ctor]). [Note: This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. —end note]~~

In 23.4.2 [pairs.pair], synopsis should use `explicit(see below)`:

```
namespace std {
    template<class T1, class T2>
    struct pair {
        pair(const pair&) = default;
        pair(pair&&) = default;
        EXPLICIT explicit(see below) constexpr pair();
        EXPLICIT explicit(see below) constexpr pair(const T1& x, const T2& y);
        template<class U1, class U2> EXPLICIT explicit(see below) constexpr pair(U1&& x, U2&& y);
        template<class U1, class U2> EXPLICIT explicit(see below) constexpr pair(const pair<U1, U2>& p);
        template<class U1, class U2> EXPLICIT explicit(see below) constexpr pair(pair<U1, U2>&& p);
        template<class... Args1, class... Args2>
            pair(piecewise_construct_t, tuple<Args1...> first_args, tuple<Args2...> second_args);
    };
}
```

In 23.4.2 [pairs.pair], paragraph 3-4:

```
EXPLICIT explicit(see below) constexpr pair();
```

Remarks: [...] ~~The constructor is explicit~~ The expression inside `explicit` evaluates to `true` if and only if either `first_type` or `second_type` is not implicitly default-constructible. [Note: This behavior can be implemented with a trait that checks whether a `const first_type&` or a `const second_type&` can be initialized with `{}`. —end note]

In 23.4.2 [pairs.pair], paragraph 5-6:

```
EXPLICIT explicit(see below) constexpr pair(const T1& x, const T2& y);
```

Remarks: [...] ~~The constructor is explicit if and only if `is_convertible_v<const first_type&, first_type>` is false or `is_convertible_v<const second_type&, second_type>` is false~~ : The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const first_type&, first_type> || !is_convertible_v<const second_type&, second_type>
```

In 23.4.2 [pairs.pair], paragraph 7-8:

```
template<class U1, class U2> EXPLICIT explicit(see below) constexpr pair(U1&& x, U2&& y);
```

Remarks: [...] ~~The constructor is explicit if and only if `is_convertible_v<U1&&, first_type>` is false or `is_convertible_v<U2&&, second_type>` is false~~ : The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U1, first_type> || !is_convertible_v<U2, second_type>
```

In 23.4.2 [pairs.pair], paragraph 9-10:

```
template<class U1, class U2> EXPLICIT explicit(see below). constexpr pair(const pair<U1, U2>& p);
```

Remarks: [...] ~~The constructor is explicit if and only if `is_convertible_v<const U1&, first_type>` is false or `is_convertible_v<const U2&, second_type>` is false.~~ The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const U1&, first_type> || !is_convertible_v<const U2&, second_type>
```

In 23.4.2 [pairs.pair], paragraph 11-12:

```
template<class U1, class U2> EXPLICIT explicit(see below). constexpr pair(pair<U1, U2>&& p);
```

Remarks: [...] ~~The constructor is explicit if and only if `is_convertible_v<U1&&, first_type>` is false or `is_convertible_v<U2&&, second_type>` is false.~~ The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U1, first_type> || !is_convertible_v<U2, second_type>
```

In 23.5.3 [tuple.tuple], synopsis should use `explicit(see below)` :

```
namespace std {
    template<class... Types>
    class tuple {
    public:
        // [tuple.cnstr], tuple construction
        EXPLICIT explicit(see below). constexpr tuple();
        EXPLICIT explicit(see below). constexpr tuple(const Types&...);           // only if sizeof...(Types) >=
        template<class... UTypes>
            EXPLICIT explicit(see below). constexpr tuple(UTypes&&...);           // only if sizeof...(Types) >=

        tuple(const tuple&) = default;
        tuple(tuple&&) = default;

        template<class... UTypes>
            EXPLICIT explicit(see below). constexpr tuple(const tuple<UTypes...>&);
        template<class... UTypes>
            EXPLICIT explicit(see below). constexpr tuple(tuple<UTypes...>&&);

        template<class U1, class U2>
            EXPLICIT explicit(see below). constexpr tuple(const pair<U1, U2>&);   // only if sizeof...(Types) ==
        template<class U1, class U2>
            EXPLICIT explicit(see below). constexpr tuple(pair<U1, U2>&&);       // only if sizeof...(Types) ==

        // allocator-extended constructors
        template<class Alloc>
            tuple(allocator_arg_t, const Alloc& a);
        template<class Alloc>
            EXPLICIT explicit(see below). tuple(allocator_arg_t, const Alloc& a, const Types&...);
        template<class Alloc, class... UTypes>
            EXPLICIT explicit(see below). tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
        template<class Alloc>
            tuple(allocator_arg_t, const Alloc& a, const tuple&);
        template<class Alloc>
            tuple(allocator_arg_t, const Alloc& a, tuple&&);
        template<class Alloc, class... UTypes>
            EXPLICIT explicit(see below). tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
        template<class Alloc, class... UTypes>
            EXPLICIT explicit(see below). tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
        template<class Alloc, class U1, class U2>
            EXPLICIT explicit(see below). tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
        template<class Alloc, class U1, class U2>
            EXPLICIT explicit(see below). tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
    };
}
```

In 23.5.3.1 [tuple.cnstr], paragraphs 5-6:

```
EXPLICIT explicit(see below). constexpr tuple();
```

Remarks: [...] ~~The constructor is explicit~~ The expression inside `explicit` evaluates to `true` if and only if `Ti` is not implicitly default-constructible for at least one *i*. [Note: This behavior can be implemented with a trait that checks whether a `const Ti&` can be initialized with `{}`. —end note]

In 23.5.3.1 [tuple.cnstr], paragraphs 7-8:

```
EXPLICIT explicit(see below). constexpr tuple(const Types&...);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<const Ti&, Ti>` is `false` for at least one *i*. The expression inside `explicit` is equivalent to:

```
!conjunction v<is_convertible<const Types&, Types>...>
```

In 23.5.3.1 [tuple.cnstr], paragraphs 9-10:

```
template<class... UTypes> EXPLICIT explicit(see below). constexpr tuple(UTypes&&...);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<Ui&&, Ti>` is `false` for at least one *i*. The expression inside `explicit` is equivalent to:

```
!conjunction v<is_convertible<UTypes, Types>...>
```

In 23.5.3.1 [tuple.cnstr], paragraphs 15-16:

```
template<class... UTypes> EXPLICIT explicit(see below). constexpr tuple(const tuple<UTypes...>& u);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<const Ui&, Ti>` is `false` for at least one *i*. The expression inside `explicit` is equivalent to:

```
!conjunction v<is_convertible<const UTypes&, Types>...>
```

In 23.5.3.1 [tuple.cnstr], paragraphs 17-18:

```
template<class... UTypes> EXPLICIT explicit(see below). constexpr tuple(tuple<UTypes...>&& u);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<Ui&&, Ti>` is `false` for at least one *i*. The expression inside `explicit` is equivalent to:

```
!conjunction v<is_convertible<UTypes, Types>...>
```

In 23.5.3.1 [tuple.cnstr], paragraphs 19-21:

```
template<class U1, class U2> EXPLICIT explicit(see below). constexpr tuple(const pair<U1, U2>& u);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<const U1&, T0>` is `false` or `is_convertible_v<const U2&, T1>` is `false`. The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const U1&, T0> || !is_convertible_v<const U2&, T1>
```

In 23.5.3.1 [tuple.cnstr], paragraphs 22-24:

```
template<class U1, class U2> EXPLICIT explicit(see below). constexpr tuple(pair<U1, U2>&& u);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<U1&&, T0>` is `false` or `is_convertible_v<U2&&, T1>` is `false`. The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U1, T0> || !is_convertible_v<U2, T1>
```

In 23.5.3.1 [tuple.cnstr], paragraphs 25:

```
template<class Alloc>
    tuple(allocator_arg_t, const Alloc& a);
template<class Alloc>
    EXPLICIT explicit(see below) tuple(allocator_arg_t, const Alloc& a, const Types&...);
template<class Alloc, class... UTypes>
    EXPLICIT explicit(see below) tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
template<class Alloc>
    tuple(allocator_arg_t, const Alloc& a, const tuple&);
template<class Alloc>
    tuple(allocator_arg_t, const Alloc& a, tuple&&);
template<class Alloc, class... UTypes>
    EXPLICIT explicit(see below) tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
template<class Alloc, class... UTypes>
    EXPLICIT explicit(see below) tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
template<class Alloc, class U1, class U2>
    EXPLICIT explicit(see below) tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
template<class Alloc, class U1, class U2>
    EXPLICIT explicit(see below) tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
```

In 23.6.3 [optional.optional], synopsis should use `explicit(see below)` :

```
template<class T>
    class optional {
    public:
        using value_type = T;

        // [optional.ctor], constructors
        constexpr optional() noexcept;
        constexpr optional(nullopt_t) noexcept;
        constexpr optional(const optional&);
        constexpr optional(optional&&) noexcept(see below);
        template<class... Args>
            constexpr explicit optional(in_place_t, Args&&...);
        template<class U, class... Args>
            constexpr explicit optional(in_place_t, initializer_list<U>, Args&&...);
        template<class U = T>
            EXPLICIT explicit(see below) constexpr optional(U&&);
        template<class U>
            EXPLICIT explicit(see below) optional(const optional<U>&);
        template<class U>
            EXPLICIT explicit(see below) optional(optional<U>&&);
    };
```

In 23.6.3.1 [optional.ctor], remove paragraph 19:

~~[Note: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. —end note]~~

In 23.6.3.1 [optional.ctor], paragraphs 20-23:

```
template<class U = T> EXPLICIT explicit(see below) constexpr optional(U&& v);
```

Remarks: [...] ~~The constructor is explicit if and only if `is_convertible_v<U&&, T>` is false~~ . The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

In 23.6.3.1 [optional.ctor], paragraphs 24-27:

```
template<class U> EXPLICIT explicit(see below) constexpr optional(const optional<U>& rhs);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<const U&, T>` ~~is false~~ : The expression inside `explicit` is equivalent to:

`!is_convertible_v<const U&, T>`

In 23.6.3.1 [optional.ctor], paragraphs 28-31:

```
template<class U> EXPLICIT explicit(see below). constexpr optional(optional<U>&& rhs);
```

Remarks: [...] ~~The constructor is explicit if and only if~~ `is_convertible_v<U&, T>` ~~is false~~ : The expression inside `explicit` is equivalent to:

`!is_convertible_v<U, T>`

Add a new subclause to Annex C:

Affected subclause: [dcl.fct]

Change: The class name can no longer be used parenthesized in an explicit constructor declaration. The *conversion-function-id* can no longer be used parenthesized in an explicit conversion function declaration.

Rationale: Necessary for new functionality.

Effect on original feature: Valid C++ 2017 code may fail to compile in this International Standard. For example:

```
struct S {  
    explicit (S)(const S&); // ill-formed; previously well-formed  
    explicit (operator int)(); // ill-formed; previously well-formed  
};
```

Acknowledgements

Thanks to Titus Winters, whose EWG Wishlist thread led to the creation of this proposal.